

Javier Romera Llave
Álvaro Rodríguez Palacios

ARQO | P3

Ejercicio 0

LEVEL1_ICACHE_SIZE	32768
LEVEL1_ICACHE_ASSOC	8
LEVEL1_ICACHE_LINESIZE	64
LEVEL1_DCACHE_SIZE	49152
LEVEL1_DCACHE_ASSOC	12
LEVEL1_DCACHE_LINESIZE	64
LEVEL2_CACHE_SIZE	524288
LEVEL2_CACHE_ASSOC	8
LEVEL2_CACHE_LINESIZE	64
LEVEL3_CACHE_SIZE	6291456
LEVEL3_CACHE_ASSOC	12
LEVEL3_CACHE_LINESIZE	64
LEVEL4_CACHE_SIZE	0
LEVEL4_CACHE_ASSOC	0
LEVEL4_CACHE_LINESIZE	0

El PC en el que estamos trabajando tiene tres niveles de memoria caché, en el primer nivel podemos observar una caché asociativa de 8 vías para las instrucciones y una caché asociativa de 12 vías para los datos, en el segundo nivel tenemos una caché asociativa de 8 vías y en el último nivel una caché asociativa de 12 vías.

Ejercicio 1

1.2) La razón principal es porque el programa que estamos utilizando no es el único programa que se está ejecutando en ese momento (al menos en nuestro entorno de trabajo), el sistema operativo que gestiona los procesos puede que haya decidido en algún momento de la ejecución de nuestro programa saltar o continuar otro proceso, es decir, está intercalando la ejecución de los distintos procesos que haya en ese momento. Y es por esto que para una misma matriz y la misma forma de recorrerla obtenemos ligeras oscilaciones en los tiempos, una buena forma de aproximar estos tiempos es usando una media aritmética.

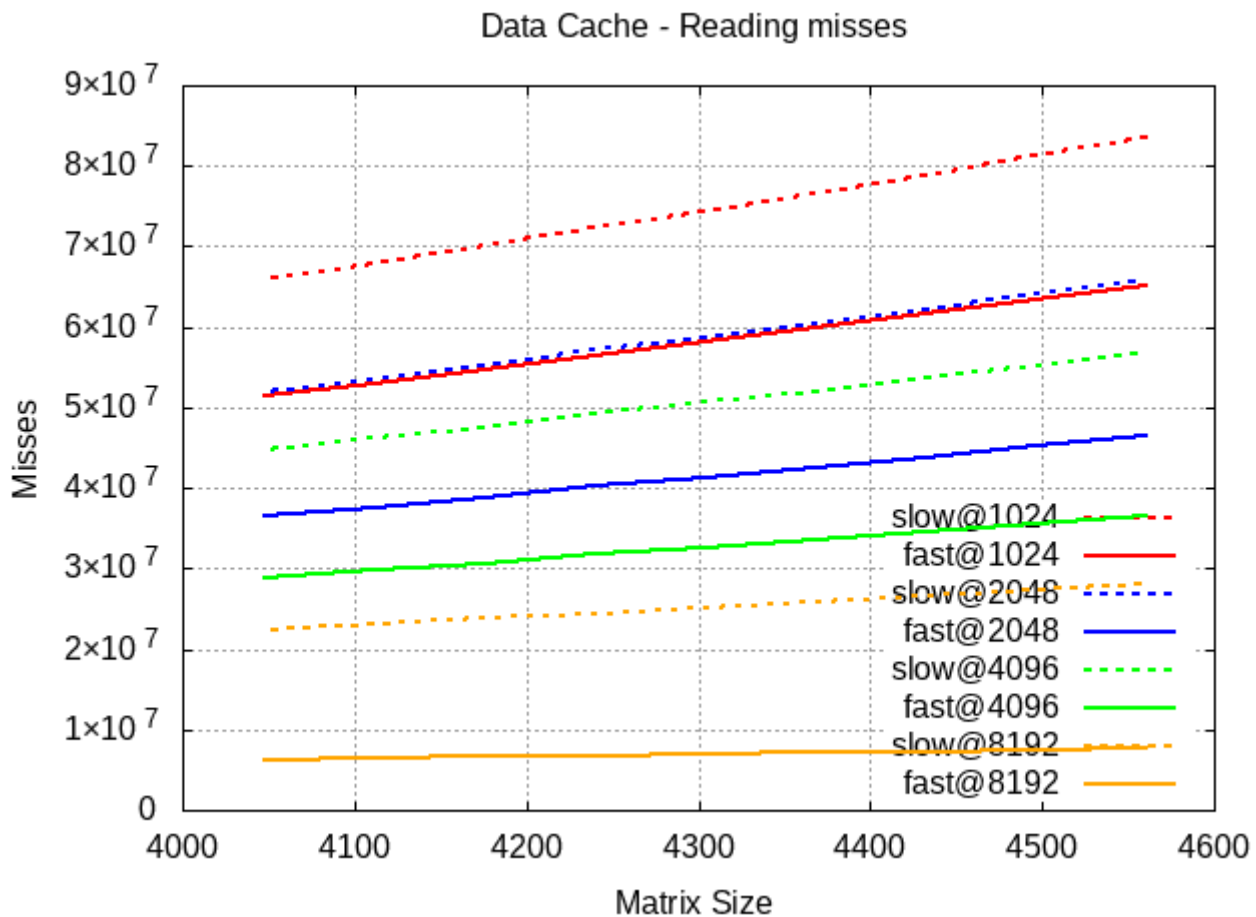
1.5) Cuando las matrices son relativamente pequeñas no se aprecia prácticamente diferencia en la forma de recorrerlas ya que al ser pequeñas caben casi completamente en la caché, pero en el momento que aumentamos el tamaño de las matrices la diferencia entre los tiempos comienza a incrementarse, y es aquí cuando influye mucho la forma en la que recorremos la matriz, si los datos los leemos de tal forma que estos estén contiguos evitamos que la caché tenga que acceder más veces a la memoria principal para volcar los bloques, en cambio, cuando accedemos de tal forma que por cada iteración accedemos a direcciones (lo suficientemente lejanas) como para que el índice del bloque de la caché se pise muchas veces, repercutirá de tal forma que los accesos a memoria principal serán mucho mayores.

La matriz se está guardando en memoria por columnas.

Ejercicio 2

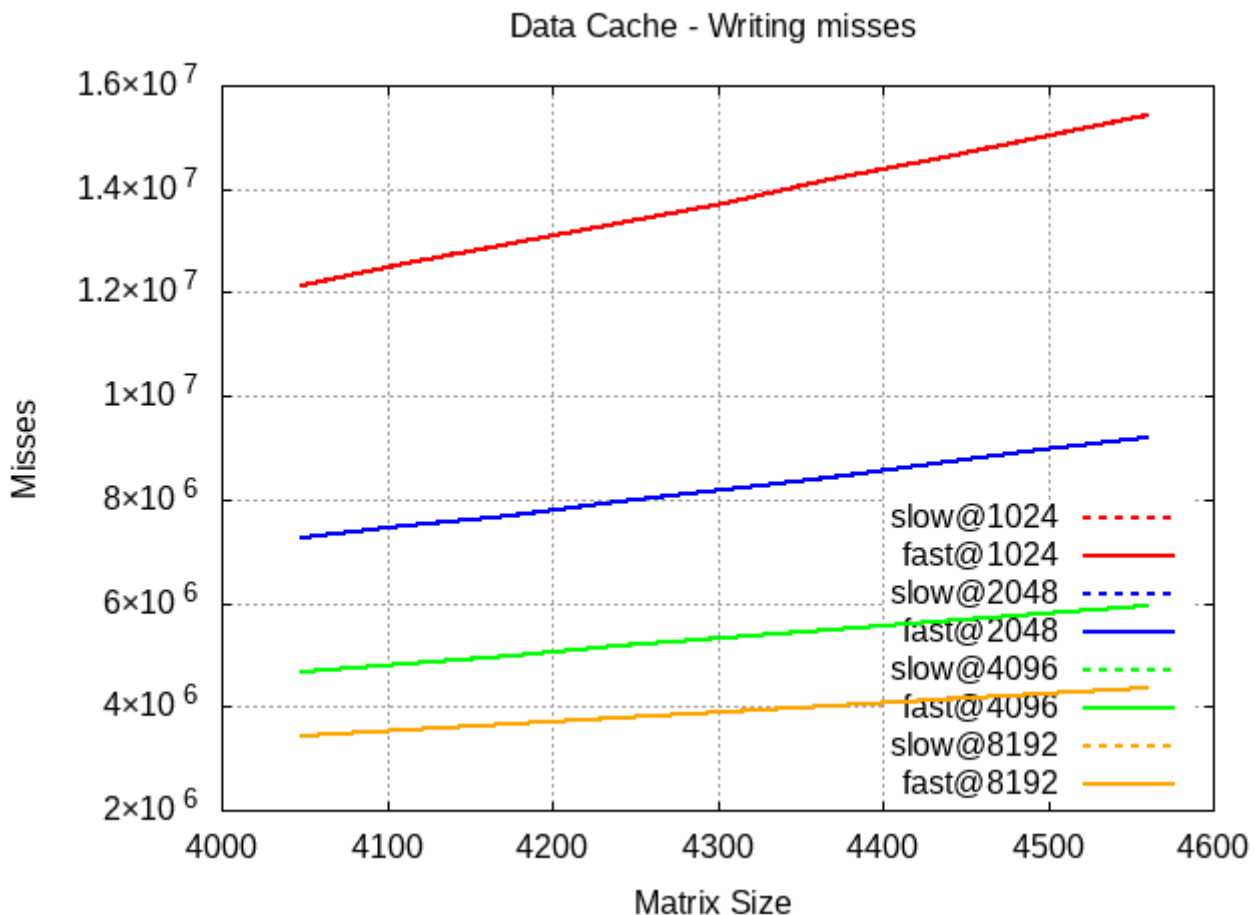
2.4)

En el caso de la versión `slow` del programa la tendencia no varía prácticamente nada al modificar el tamaño de la caché, es decir, la pendiente de las rectas que forman los datos son casi siempre iguales, en cambio en la versión `fast` del programa la tendencia de estas rectas si varía bastante más, llegando a aplanarse bastante como podemos apreciar en el siguiente gráfico:



Si ahora comparamos las dos versiones fijándonos en un tamaño específico de la caché, podemos observar que la versión `fast` del programa siempre está mucho más por debajo de la versión `slow` y con una pendiente mucho más suave.

Si observamos el gráfico de escrituras:



Se puede apreciar que las rectas de la versión `slow` y `fast` están superpuestas, para ver el por qué de esto podemos fijarnos en el análisis de la emulación con el comando `cg_annotate` :

```

Il cache: 1024 B, 64 B, direct-mapped
Dl cache: 1024 B, 64 B, direct-mapped
Ll cache: 8388608 B, 64 B, direct-mapped
Command: ./fast 4304
Data file: cachegrind.fast.1024
Events recorded: Ir I1mr ILmr Dr D1mr DLmr Dw D1mw DLmw
Events shown: Ir I1mr ILmr Dr D1mr DLmr Dw D1mw DLmw
Event sort order: Ir I1mr ILmr Dr D1mr DLmr Dw D1mw DLmw
Thresholds: 0.1 100 100 100 100 100 100 100 100
Include dirs:
User annotated:
Auto-annotation: off

-----
Ir      I1mr    ILmr  Dr      D1mr    DLmr    Dw      D1mw    DLmw
-----
1,831,887,749 74,103,954 1,927 666,996,346 58,092,681 2,320,074 166,746,787 13,737,056 2,318,552  PROGRAM TOTALS

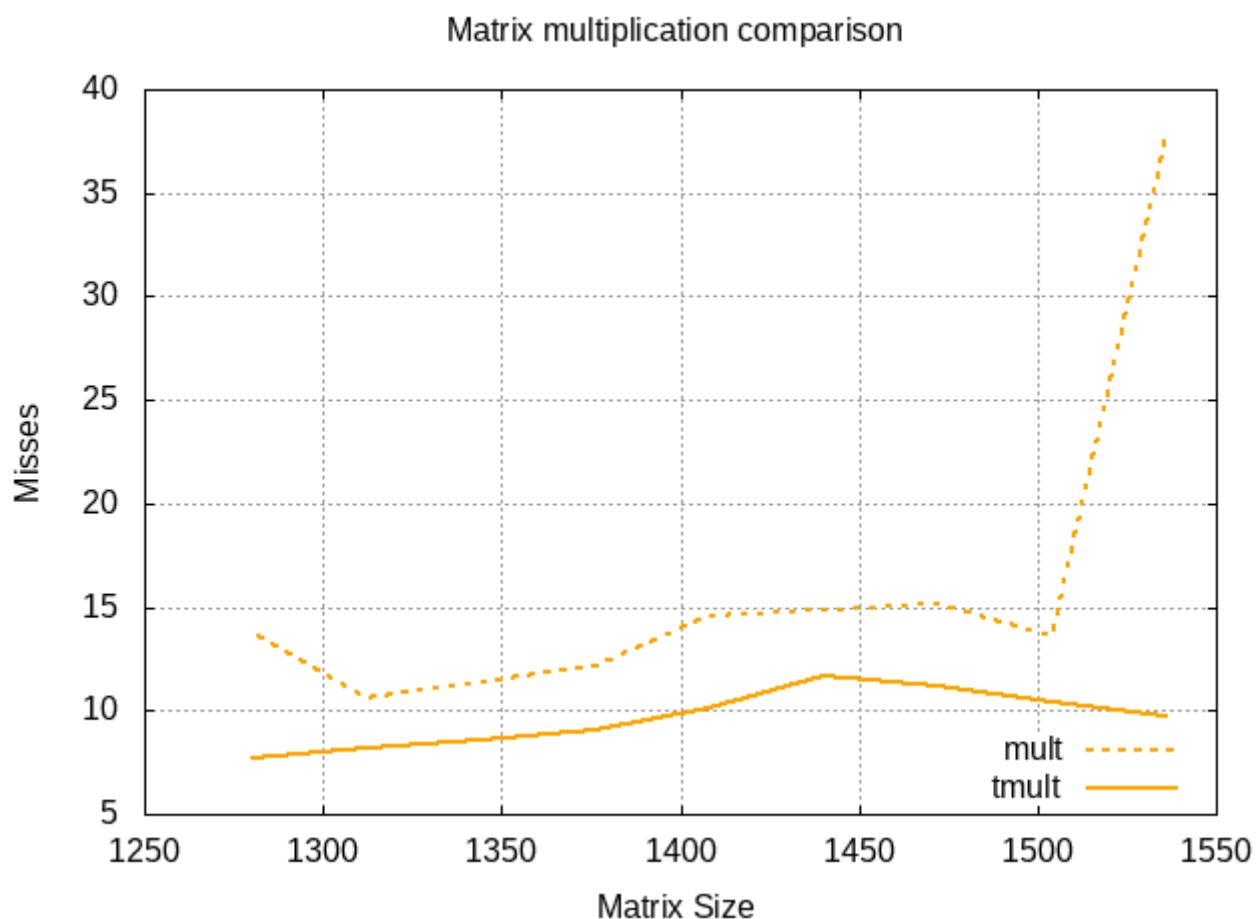
-----
Ir      I1mr    ILmr  Dr      D1mr    DLmr    Dw      D1mw    DLmw    file:function
-----
590,391,068 18,524,418 139 148,195,328 14,426,553 311 55,573,248 0 0 /build/glibc-ZN95T4/glibc-2.31/stdlib/random_r.c:random_r
389,012,736 18,524,418 138 148,195,328 13,003,338 136 37,048,832 2,223,049 136 /build/glibc-ZN95T4/glibc-2.31/stdlib/random.c:random
352,054,326 18,524,424 142 148,238,378 17,308,678 545 37,057,452 11,511,016 2,317,778 /home/lromeraj/uan/c3/arqo/p3/arqo3.c:generateMatrix
333,478,237 3 3 166,741,269 8,861,868 2,316,602 18,528,724 0 0 /home/lromeraj/uan/c3/arqo/p3/fast.c:compute
92,622,080 1 1 18,524,416 0 0 18,524,416 0 0 /build/glibc-ZN95T4/glibc-2.31/stdlib/rand.c:rand
37,049,113 18,524,500 176 18,524,528 2,255,401 146 15 2 1 ???
37,048,832 0 0 18,524,416 2,223,049 136 0 0 0 /build/glibc-ZN95T4/glibc-2.31/stdlib/./sysdeps/unix/sysv/linux/x86/lowlevelloc
k.h:random

```

Se puede apreciar que la función `compute` que es la que suma todos los elementos de la matriz, no ha provocado ningún fallo de escritura en la caché de datos de nivel 1. Dado que ninguna de las dos versiones de la función realiza la acción de escribir en memoria principal y el resto del programa es idéntico no de aprecian diferencias entre ambas versiones.

Ejercicio 3

3.5) Viendo la gráfica del tiempo:



Podemos ver que el segundo método (usando la matriz traspuesta) es mucho más eficiente. La gráfica presenta múltiples picos debido a que las pruebas no se realizaron el suficiente número de veces, pero aún así se puede ver como las variaciones son mucho menores usando la matriz traspuesta.